【其他】

零、某些算法（?

1、通过给予开始结束年月，输出区间年月

2、

3、

4、

5、

6、

7、

8、

9、

一、JSON 返回类

二、java 执行 HTTP 连接

1、配置文件

2、java 代码

三、java 使用 ssh 连接 linux 执行命令

1、maven

2、代码

四、将 url 中的特殊字符解码

五、使用 fastjson 框架，将 json 字符串反序列化并转为 Map 集合

1、maven

2、代码

六、maven 父子项目设置

六、

七、

八、

九、

十、

十一、

十二、

十三、

十四、

十五、

十六、

十七、

十八、

十九、

二十、

1、

2、

3、

4、

5、

6、

7、

8、

9、

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、
Ⅵ、
Ⅶ、
Ⅷ、
Ⅸ、
Ⅹ、

> 阿里 java 开发规约：📎java开发规约.pdf

零、某些算法（?

1、通过给予开始结束年月，输出区间年月

Java 复制代码

```java
package com.yuehai.redis_springboot_02.controller;

import com.yuehai.redis_springboot_02.utils.JsonResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;

/**
 * @author 月海
 * @create 2023/2/10 9:18
 */

@RestController
public class yearController {


    @GetMapping("/year")
    public JsonResult year(){
        // 开始年
        int startYear = 2022;
        // 开始月
        int startMth = 2;

        // 结束年
        int endYear = 2024;
        // 结束月
        int endMth = 8;

        // 当前年，初始值为选中的开始年
        int theYear = startYear;
        // 当前月，初始值为选中的开始月
        int theMth = startMth;

        // 保存的集合
        List<String> year = new ArrayList<>();

        // 循环年
        for( theYear = startYear; theYear <= endYear; theYear++ ){

            // 循环月
            for ( ; theMth<= 12; theMth++ ){
                // 判断当前月是几位数，以判断加不加 0
                if (theMth < 10){
```

```java
                    // 添加到集合中，返回页面
                    year.add(theYear+ "0" + theMth);

                    // 判断本次循环后是否是选中的结束年月，是则跳出循环
                    if ( (theYear+ "0" + theMth).equals(endYear+ "0" + endMth) ){
                        break;
                    }
                }else {
                    year.add(String.valueOf(theYear) + theMth);

                    if ( (String.valueOf(theYear) + theMth).equals(String.valueOf(endYear) + endMth) ){
                        break;
                    }
                }
            }

            theMth = 1;
            year.add("--------");

        }

        return JsonResult.success(year);
    }

    @GetMapping("/year2")
    public JsonResult year2(){

        // 开始年月
        int startYear = 202106;
        // 结束年月
        int endYear = 202310;

        // 保存的集合
        List<Integer> year = new ArrayList<>();

        // 循环
        for ( ; startYear <= endYear; startYear++ ){
            year.add(startYear);

            if ( ( startYear % 100 ) == 12 ){
                startYear = ( startYear / 100 + 1 ) * 100;
            }
        }

        return JsonResult.success(year);
    }
```

```
}
```

2、

3、

4、

5、

6、

7、

8、

9、

一、JSON 返回类

JsonResult　　Java　复制代码

```java
package com.yuehai.redis_springboot_02.utils;

import java.io.Serializable;
import java.util.HashMap;
import java.util.Map;

/**
 * @author 月海
 * @create 2022/9/20 15:20
 *
 * 使用 @RestController 或 @ResponseBody 时，可直接返回该对象，
 * Spring Boot 默认使用 Jackson 会自动将该对象转换为 json 字符串
 * 如 {"code": 0,"msg": "","data": [{}, {}]}
 */
public class JsonResult extends HashMap<String, Object> implements Serial
izable {
    private static final long serialVersionUID = 1L;
    /**
     * 成功
     */
    private static final int SUCCESS = 200;
    /**
     * 失败
     */
    private static final int FAIL = 1;
    /**
     * 错误
     */
    private static final int ERROR = 2;
    /**
     * 未登录
     */
    private static final int LOGOUT = 1001;

    /**
     * 构造器
     * @param code 状态码
     * @param msg 提示消息
     * @param data 数据
     */
    public JsonResult(int code, String msg, Object data) {
        // 继承自 Map，设置初始容量
        super(3);
        // 状态码，layui 中 code=0 表示成功
        this.put("code", code);
```

```java
        // 提示消息
        this.put("msg", msg);
        // 数据体
        this.put("data", data);
    }
    /**
     * 构造器
     * @param code 状态码
     * @param msg 提示消息
     */
    public JsonResult(int code, String msg) {
        // 状态码
        this.put("code", code);
        // 提示消息
        this.put("msg", msg);
    }

    /**
     * 一般返回 code、msg 和 data 这三个即可，但 layui 加载 table 时还要求 count 值
     * 添加额外的返回值
     * @param key 额外信息 - key
     * @param value 额外信息 - value
     * @return 返回 额外信息
     */
    @Override
    public JsonResult put(String key, Object value) {
        super.put(key, value);
        return this;
    }


    /**
     * 快速返回请求成功
     * @param data 数据
     * @return 快速返回请求成功
     */
    public static JsonResult success(Object data) {
        return new JsonResult(SUCCESS, "ok", data);
    }

    /**
     * 快速返回请求失败
     * @param msg 提示消息
     * @return 快速返回请求失败
     */
    public static JsonResult fail(String msg) {
        return new JsonResult(FAIL, msg, null);
```

```java
    }

    /**
     * 快速返回请求错误
     * @param msg 提示消息
     * @param data 数据
     * @return 快速返回请求失败
     */
    public static JsonResult error(String msg, Object data) {
        return new JsonResult(ERROR, msg, data);
    }
    /**
     * 快速返回请求错误 - 自定义状态码和错误信息
     * @param code 状态码
     * @param msg 提示消息
     * @return 快速返回请求错误 - 自定义状态码和错误信息
     */
    public static JsonResult error(int code, String msg) {
        return new JsonResult(code, msg);
    }

    /**
     * 快速返回未登录
     * @return 快速返回未登录
     */
    public static JsonResult logout() {
        return new JsonResult(LOGOUT, "未登录", null);
    }

    /**
     * 快速生成一个 Map 键值对
     * @param key 额外信息 - key
     * @param value 额外信息 - value
     * @return 快速生成一个 Map 键值对
     */
    public static Map<String, Object> fastMap(String key, Object value) {
        Map<String, Object> data = new HashMap<>(1);
        data.put(key, value);
        return data;
    }
}
```

二、java 执行 HTTP 连接

1、配置文件

YAML | 复制代码

```yaml
# yaml 配置文件
spring:
  datasource:
    # 连接信息一定要用双引号引起来
    # characterEncoding=UTF-8：指定所处理字符的解码和编码的格式
    # useSSL=false：不进行 SSL 连接
    # allowPublicKeyRetrieval=true：允许恶意代理执行MITM攻击以获取明文密码，因此它在默认情况下为False，必须显式启用
    # serverTimezone=GMT：格林尼治标准时间
    url: "jdbc:mysql://172.20.2.55:3335/edu-trhpir-c?characterEncoding=UTF-8&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=GMT"
    username: "root"
    password: "123456"
    # 数据库驱动名
    driver-class-name: com.mysql.cj.jdbc.Driver

# 配置 MyBatis 规则
mybatis:
  # sql 映射文件位置
  mapper-locations: "classpath:mappers/*.xml"
  configuration:
    # 是否开启自动驼峰命名规则映射
    map-underscore-to-camel-case: true
    # 开启 mybatis 的 sql 执行日志打印，可以在控制台打印输出 sql 语句
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl

# 配置端口号
server:
  port: 8080

# 通用 mapper 配置
mapper:
  # 在进行数据库操作的时候，判断一个属性是否为空的时候，是否需要自动追加不为空字符串的判断；如：username != null 不需要 username != ''
  not-empty: false

# CGI 属性配置
CGI:
  CGI: "http://172.20.2.55:9033/~trial/TRHPIR/CGI/WEB_TRHPIR.CGI"
  Download: "http://172.20.2.55:9033/~trial/TRHPIR/DOWNLOAD/"
```

2、java 代码

Java 复制代码

```java
package com.trial.edutrhpircback.utils;

import lombok.Data;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;

/**
 * @author 月海
 * @create 2022/10/12 15:24
 */

@Component
@Data
public class CGIUtil {
    // SEARCH 调用 CGI
    @Value("${CGI.CGI}")
    public String CGI;
    // 下载路径
    @Value("${CGI.Download}")
    public String Download;

    public String UTF8 = "UTF-8";

    /**
     * get 请求
     * @param param 参数
     * @return 响应数据
     * @throws IOException
     */
    public String getCgi(String param) {
        // 接收返回的数据
        StringBuilder result = new StringBuilder();

        HttpURLConnection connection = null;
        InputStream inputStream = null;
        BufferedReader bufferedReader = null;

        try {
            // 创建连接
            URL url = new URL(CGI + "?" + param);
            connection = (HttpURLConnection) url.openConnection();
```

```
            // 设置请求方式
            connection.setRequestMethod("GET");
            // 设置连接超时时间
            connection.setReadTimeout(15000);

            // 开始连接
            connection.connect();

            // 获取响应数据
            if ( connection.getResponseCode() == 200 ){
                // 获取返回的数据
                inputStream = connection.getInputStream();
                if (null != inputStream){
                    bufferedReader = new BufferedReader(new InputStreamReader(inputStream, UTF8));
                    String temp = null;
                    while (null != (temp = bufferedReader.readLine())){
                        result.append(temp).append("\r\n");
                    }
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            // 关闭资源
            if (null != bufferedReader){
                try {
                    bufferedReader.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            if (null != inputStream){
                try {
                    inputStream.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }

            if (null != connection){
                // 关闭连接
                connection.disconnect();
            }
        }
        // 返回接收的数据
        return result.toString();
    }
```

```java

    /**
     * post 请求
     * @param param 参数
     * @return 响应数据
     * @throws IOException
     */
    public String postCgi(String param) {
        // 接收返回的数据
        StringBuilder result = new StringBuilder();

        HttpURLConnection connection = null;
        DataOutputStream dataOutputStream = null;
        InputStream inputStream = null;
        BufferedReader bufferedReader = null;
        try {
            // 创建连接
            URL url = new URL(CGI);
            connection = (HttpURLConnection) url.openConnection();
            // 设置请求方式
            connection.setRequestMethod("POST");
            // 设置连接超时时间
            connection.setConnectTimeout(15000);
            //设置读取超时时间
            connection.setReadTimeout(15000);

            // 设置是否可读取；doOutput 设置是否向httpUrlConnection输出
            connection.setDoOutput(true);
            //  doInput 设置是否从httpUrlConnection读入，发送post请求必须设置这两个
            connection.setDoInput(true);

            //设置通用的请求属性
            connection.setRequestProperty("accept", "*/*");
            connection.setRequestProperty("connection", "Keep-Alive");
            connection.setRequestProperty("user-agent", "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1)");
            connection.setRequestProperty("Content-Type", "application/json;charset=utf-8");

            // 设置权限
            // 设置请求头等

            // 开启连接；建立TCP连接，getOutputStream 会隐含的进行connect，所以此处可以不要
            // 创建输入输出流,用于往连接里面输出携带的参数
            dataOutputStream = new DataOutputStream(connection.getOutputStream());
```

```java
            // 设置参数
            dataOutputStream.write(param.getBytes(UTF8));
            dataOutputStream.flush();
            dataOutputStream.close();

            // 获取响应数据
            if ( connection.getResponseCode() == 200 ){
                // 获取返回的数据
                inputStream = connection.getInputStream();
                if (null != inputStream){
                    bufferedReader = new BufferedReader(new InputStreamReader(inputStream, UTF8));
                    String temp = null;
                    while (null != (temp = bufferedReader.readLine())){
                        result.append(temp).append("\r\n");
                    }
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            // 关闭资源
            if (null != bufferedReader){
                try {
                    bufferedReader.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            if (null != inputStream){
                try {
                    inputStream.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            if (null != dataOutputStream){
                try {
                    dataOutputStream.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            if (null != connection){
                // 关闭连接
                connection.disconnect();
            }
        }
```

```
            // 返回接收的数据
            return result.toString();
        }


    }
```

三、java 使用 ssh 连接 linux 执行命令

1、maven

XML 复制代码

```xml
<!-- ssh -->
<dependency>
    <groupId>ch.ethz.ganymed</groupId>
    <artifactId>ganymed-ssh2</artifactId>
    <version>262</version>
</dependency>
```

2、代码

Java 复制代码

```java
import javax.servlet.ServletOutputStream;
import javax.servlet.http.HttpServletResponse;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.*;

/**
 * @author 月海
 * @create 2022/9/21 13:18
 */

@Data
@NoArgsConstructor
@Slf4j
@Component
public class SSHCommandutil {

    public static final String IP = "172.20.2.55";
    public static final String USERNAME = "creat";
    public static final String USERPWD = "creat1234";

    /**
     * 远程登录 linux 的主机；接收会话信息
     * @param conn
     * @return
     */
    public Boolean login(Connection conn) {
        boolean flg = false;

        // 连接
        try {
            conn.connect();
        } catch (IOException e) {
            conn.close();
            e.printStackTrace();

        }

        // 认证
        try {
            flg = conn.authenticateWithPassword(USERNAME, USERPWD);
        } catch (IOException e) {
            conn.close();
            e.printStackTrace();
        }


        return flg;
```

```java
    }

    // 远程执行 shll 脚本或者命令
    public StringBuilder execute(String... args) {
        StringBuilder buffer = null;
        Session session = null;

        // 创建会话信息，传递给 login 方法
        Connection conn = new Connection(IP);
        if (login(conn)){

            try {
                // 打开一个会话
                session = conn.openSession();
                // 建立虚拟终端
                session.requestPTY("bash");
            } catch (IOException e) {
                e.printStackTrace();
            }

            // 打开一个Shell
            try {
                session.startShell();
            } catch (IOException e) {
                e.printStackTrace();
            }

            // 准备输入命令
            PrintWriter out = new PrintWriter(session.getStdin());
            // 输入待执行命令
            out.println("cd /home/creat/workspace/trhpir_c_pj");
            out.println("make");
            // 遍历传入的可变参数，循环执行命令
            for (String cmd : args) {
                out.println(cmd);
            }
            // 关闭 linux 端的会话
            out.println("exit");
            out.flush();

            // 关闭输入流
            out.close();

            // 调用方法，得到返回结果
            buffer = processStdout(session.getStdout());
        }

        // 关闭会话
```

```java
        session.close();
        conn.close();

        return buffer;
    }

    // 解析脚本执行返回的结果集，以纯文本的格式返回
    private StringBuilder processStdout(InputStream in) {
        // 定义接收数据的字符串
        StringBuilder buffer = new StringBuilder();

        // 包装输入流
        BufferedReader br = new BufferedReader(new InputStreamReader(in, StandardCharsets.UTF_8));

        // 创建线程，定义线程数为 1
        ExecutorService es = Executors.newFixedThreadPool(1);
        // Future用于执行多线程的执行结果
        BufferedReader finalBr = br;
        Future future = es.submit( () -> {
            String line;
            try {
                while ( (line = finalBr.readLine()) != null){
                    // 将传输过来的参数赋值给 buffer
                    buffer.append(line + "\n");
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        } );

        // 线程运行时间为一秒，超时退出
        try {
            future.get(1, TimeUnit.SECONDS);
        } catch (Exception e) {
            System.out.println("数据抽出完成，退出线程，关闭会话");
        }

        return buffer;
    }

    // 下载文件
    public void getFile(String filePath, HttpServletResponse response) {
        // 创建会话信息，传递给 login 方法
        Connection conn = new Connection(IP);
        if (login(conn)){
            try {
                SCPClient scpClient = conn.createSCPClient();
```

```
                // 读取文件
                SCPInputStream scpInputStream = scpClient.get(filePath);

                byte[] bytes = StreamUtils.copyToByteArray(scpInputStream);

                // 文件名
                String fileName = "1111.xlsx";

                response.reset();
                response.setContentType("application/octet-stream");
                response.addHeader("Content-Disposition", "attachment;fileName=" + new String(fileName.getBytes(StandardCharsets.UTF_8), StandardCharsets.ISO_8859_1));
                response.addHeader("Content-Length", "" + bytes.length);

                ServletOutputStream outputStream = response.getOutputStream();
                outputStream.write(bytes);
                outputStream.flush();
            } catch (IOException e) {
                e.printStackTrace();
            }

        }

    }
}
```

```
195
196
197
198
199
200
201
```

四、将 `url` 中的特殊字符解码

1. `url` 字符串：`%7B%22name%22%3A%22userid%22%2C%22value%22%3A%2210222148%22%2C%22expires%22%3A1%2C%22path%22%3A%22%2F%22%7D`
2. 解码后：`{"name":"userid","value":"10222148","expires":1,"path":"/"}`

JavaScript | 复制代码

```java
package com.yuehai;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

import java.io.UnsupportedEncodingException;
import java.net.URLDecoder;

/**
 * @author 月海
 * @create 2022/12/8 8:19
 */

@SpringBootTest
class URLDecoderDecode {

    @Test
    void test01() throws UnsupportedEncodingException {
        String str = "%7B%22name%22%3A%22userid%22%2C%22value%22%3A%2210222148%22%2C%22expires%22%3A1%2C%22path%22%3A%22%2F%22%7D";

        // 将url 编码解码:  {"name":"userid","value":"10222148","expires":1,"path":"/"}
        String decode = URLDecoder.decode(str, "UTF-8");

        System.out.println(decode);
    }

}
```

测试 已通过: 1共 1 个测试 – 259毫秒

```
{"name":"userid","value":"10222148","expires":1,"path":"/"}
```

五、使用 `fastjson` 框架，将 `json` 字符串反序列化并转为 `Map` 集合

1. 使用 `fastjson` 框架，将 `json` 字符串反序列化
2. 将 `json` 字符串转化为 `map` 集合

1、maven

XML | 复制代码

```xml
<!-- https://mvnrepository.com/artifact/com.alibaba.fastjson2/fastjson2 -->
<dependency>
    <groupId>com.alibaba.fastjson2</groupId>
    <artifactId>fastjson2</artifactId>
    <version>2.0.20</version>
</dependency>
```

2、代码

Java 复制代码

```java
package com.yuehai;

import com.alibaba.fastjson2.JSON;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

import java.io.UnsupportedEncodingException;
import java.net.URLDecoder;
import java.util.Map;
import java.util.function.BiConsumer;

/**
 * @author 月海
 * @create 2022/12/8 8:19
 */

@SpringBootTest
class URLDecoderDecode {

    @Test
    void test01() throws UnsupportedEncodingException {
        String str = "%7B%22name%22%3A%22userid%22%2C%22value%22%3A%2210222148%22%2C%22expires%22%3A1%2C%22path%22%3A%22%2F%22%7D";

        // 将url 编码解码:
        // {"name":"userid","value":"10222148","expires":1,"path":"/"}
        String decode = URLDecoder.decode(str, "UTF-8");

        // 使用 fastjson 框架, 将 json 字符串反序列化:
        // {"name":"userid","value":"10222148","expires":1,"path":"/"}
        Object parse = JSON.parse(decode);
        // 将 json 字符串转化为 map 集合
        // {"name":"userid","value":"10222148","expires":1,"path":"/"}
        Map<String, Object> map = (Map<String, Object>) parse;

        // 从 map 集合中使用 key 取出值
        System.out.println("name --> "+ map.get("name"));
        System.out.println("value --> " + map.get("value"));
        System.out.println("--------------------------------");

        // Lamubda 表达式遍历
        map.forEach(new BiConsumer<String, Object>() {
            @Override
            public void accept(String key, Object value) {
                System.out.println(key + "\t --> \t" + value);
```

```java

            }
        });
        System.out.println("--------------------------------");

        /**
         * 简化 Lamubda 表达式遍历
         * 化简的规则为:
         * 参数类型可以省略不写
         * 如果只有一个参数，参数类型可以省略，同时参数也可以省略
         * 如果Lamubda表达式的方法体代码只有一行代码。可以省略大括号不写，同时省略分号
         * 如果Lamubda表达式的方法体只有一行代码。可以省略大括号不写。此时，如果这行代码是 return 语句，必须省略 return 不写，同时也必须省省略分号不写
         */
        map.forEach((key, value)-> {
            System.out.println(key + "\t --> \t" + value);
        });
    }

}
```

六、maven 父子项目设置

1. 父项目 pom.xml 文件

父项目 pom.xml 文件 XML 复制代码

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <!-- Spring Boot 项目，所以需要继承 spring-boot-starter-parent -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.6.11</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <!-- 本项目的设置 -->
    <groupId>com.yuehai</groupId>
    <artifactId>SpringBoot_redis</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>SpringBoot_redis</name>
    <description>SpringBoot_redis</description>
    <!--  packaging 的默认打包类型是 jar，所有的父工程打包方式都需要设置成 pom -->
    <packaging>pom</packaging>

    <!-- 子项目 -->
    <modules>
        <module>01_Jedis_Test</module>
        <module>02_Jedis_VerificationCode</module>
        <module>03_redis_SpringBoot</module>
    </modules>

    <!-- jdk 版本 -->
    <properties>
        <java.version>1.8</java.version>
    </properties>

    <!-- 依赖配置：目前这里的配置的依赖所引入的jar包在此工程下的所有子工程都会被引入 -->
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <!-- maven 插件 -->
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
                <configuration>
                    <source>1.8</source>
                    <target>1.8</target>
                    <encoding>UTF-8</encoding>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

2. 子项目 pom.xml 文件

子项目 pom.xml 文件 XML 复制代码

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <!-- 指定继承的父项目 -->
    <parent>
        <!-- 父项目: groupId 组 -->
        <groupId>com.yuehai</groupId>
        <!-- 父项目: artifactId 名 -->
        <artifactId>SpringBoot_redis</artifactId>
        <!-- 父项目: version 版本 -->
        <version>0.0.1-SNAPSHOT</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <!-- 本项目的设置 -->
    <artifactId>redis_SpringBoot</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>03_redis_SpringBoot</name>
    <description>03_redis_SpringBoot</description>

    <!-- 本项目所需的独有依赖 -->
    <dependencies>
        <!-- redis -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>

        <!-- spring2.X 集成 redis 所需 common-pool2 -->
        <dependency>
            <groupId>org.apache.commons</groupId>
            <artifactId>commons-pool2</artifactId>
            <version>2.11.1</version>
        </dependency>
    </dependencies>

</project>

```

六、

七、

八、

九、

十、

十一、

十二、

十三、

十四、

十五、

十六、

十七、

十八、

十九、

二十、

1、

2、

3、

4、

5、

6、

7、

8、

9、

---

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

Ⅸ、

Ⅹ、